Manual Answers: Binary Arithmetic

Q1. Signed integer addition using 8-bit 2's complement

Range of 8-bit 2's complement: −128 to +127. Positive numbers are written in ordinary binary with leading zeros. For a negative number, invert the bits of its positive magnitude and add 1.

(a) (+20) + (+15)

```
+20 = 00010100
+15 = 00001111

   00010100
 + 00001111
 -----------
   00100011

00100011 = 32 + 2 + 1 = 35
Answer: 00100011■ = +35■■
Overflow: No. The result is within −128 to +127.
```

(b) (−20) + (−15)

```
First convert each negative number to 8-bit 2's complement:

+20 = 00010100  -> invert: 11101011 -> add 1: 11101100
+15 = 00001111  -> invert: 11110000 -> add 1: 11110001

   11101100
 + 11110001
 -----------
 1 11011101

Discard the carry out of the 8-bit result:
Result = 11011101

Find its magnitude: invert 11011101 -> 00100010; add 1 -> 00100011 = 35.
Since the sign bit is 1, result = −35.
Answer: 11011101■ = −35■■
Overflow: No. Adding two negative numbers gives a negative result in range.
```

Python program for Q1

```
def to_8bit(n):
    return format(n & 0xFF, "08b")

def add_8bit(a, b):
    result = (a + b) & 0xFF
    # Overflow if same-sign operands produce an opposite-sign result
    overflow = ((a >= 0 and b >= 0 and result >= 128) or
                (a < 0 and b < 0 and result < 128))
    signed_result = result if result < 128 else result - 256
    return result, signed_result, overflow

for a, b in [(20, 15), (-20, -15)]:
    raw, decimal, overflow = add_8bit(a, b)
    print(f"{a:+d} + ({b:+d})")
    print(f"  A = {to_8bit(a)}")
    print(f"  B = {to_8bit(b)}")
    print(f"  Result = {format(raw, '08b')} = {decimal}")
    print(f"  Overflow = {'Yes' if overflow else 'No'}")
```

Expected output

```
+20 + (+15)
  A = 00010100
```

```
  B = 00001111
  Result = 00100011 = 35
  Overflow = No
-20 + (-15)
  A = 11101100
  B = 11110001
  Result = 11011101 = -35
  Overflow = No
```

Q2. Floating-point subtraction: 1.5 − 0.25

Step 1: Convert each decimal number to binary

```
1.5 = 1 + 0.5 = 1.1■
0.25 = 1/4 = 0.01■
```

Step 2: Align the binary points

```
  1.10■
- 0.01■
--------
  1.01■
```

Step 3: Verify by place values

```
1.10■ = 1 + 1/2 = 1.5
0.01■ = 1/4 = 0.25
1.01■ = 1 + 1/4 = 1.25

Therefore, 1.5 − 0.25 = 1.25
Final answer: 1.01■ = 1.25■■
```

Python program for Q2

```
a = 1.5
b = 0.25
result = a - b

print("1.5 in binary: 1.1")
print("0.25 in binary: 0.01")
print("Subtraction: 1.10 - 0.01 = 1.01")
print("Result in decimal:", result)
```

Expected output

```
1.5 in binary: 1.1
0.25 in binary: 0.01
Subtraction: 1.10 - 0.01 = 1.01
Result in decimal: 1.25
```

Note: Q2 shows the binary arithmetic manually and uses Python to verify the decimal result. This is a simple exact binary example; it does not implement a full IEEE-754 floating-point datapath.

```python
def to_twos_comp_8bit(val: int) -> int:
    return val & 0xFF
def to_binary_str(val: int) -> str:
    return f"{to_twos_comp_8bit(val):08b}"
def to_signed_decimal(val: int) -> int:
    val = val & 0xFF
    return val - 256 if val >= 128 else val
def add_8bit(a_dec: int, b_dec: int):
    a_bin = to_twos_comp_8bit(a_dec)
    b_bin = to_twos_comp_8bit(b_dec)
    carry = 0
    carry_into_msb = 0
    carry_out_msb = 0
    result_bits = []
    for i in range(8):
        bit_a = (a_bin >> i) & 1
        bit_b = (b_bin >> i) & 1
        if i == 7:
            carry_into_msb = carry
        s = bit_a ^ bit_b ^ carry
        carry = (bit_a & bit_b) | (carry & (bit_a ^ bit_b))
        result_bits.append(str(s))
    carry_out_msb = carry
    result_bin = int("".join(reversed(result_bits)), 2)
    overflow = bool(carry_into_msb ^ carry_out_msb)
    return {
        "op_a_bin": to_binary_str(a_bin),
        "op_b_bin": to_binary_str(b_bin),
        "res_bin": to_binary_str(result_bin),
        "res_dec": to_signed_decimal(result_bin),
        "expected_dec": a_dec + b_dec,
        "carry_out": carry_out_msb,
        "overflow": overflow,
    }
def sub_8bit(a_dec: int, b_dec: int):
    return add_8bit(a_dec, -b_dec)
def display_verification(label: str, a: int, b: int, is_subtraction: bool = False):
    symbol = "-" if is_subtraction else "+"
    res = sub_8bit(a, b) if is_subtraction else add_8bit(a, b)

    print(f"=== Expression {label}: ({a:+d}) {symbol} ({b:+d}) ===")
    print(f"  Operand A:  {res['op_a_bin']} ({a:+d})")
    print(f"  Operand B:  {res['op_b_bin']} ({b:+d})")
    print(f"  Carry-Out:  {res['carry_out']}")
    print(f"  Result Bin: {res['res_bin']}")
    print(f"  Result Dec: {res['res_dec']}")
    print(f"  Overflow:   {'YES (Arithmetic Overflow)' if res['overflow'] else 'NO'}")
    print()
display_verification("(a)", 20, 15)
display_verification("(b)", -20, -15)
```

```
iscellaneous programs/1"
=== Expression (a): (+20) + (+15) ===
  Operand A:  00010100 (+20)
  Operand B:  00001111 (+15)
  Carry-Out:  0
  Result Bin: 00100011
  Result Dec: 35
  Overflow:   NO

=== Expression (b): (-20) + (-15) ===
  Operand A:  11101100 (-20)
  Operand B:  11110001 (-15)
  Carry-Out:  1
  Result Bin: 11011101
  Result Dec: -35
  Overflow:   NO

(base) vinaya@VINAYAs-MBP miscellaneous programs
% 
```

```python
def to_binary_8bit(val: int) -> str:
    return f"{val & 0xFF:08b}"
def logical_left_shift_8bit(bin_str: str, positions: int = 2):
    print(f"--- Logical Left Shift (<< {positions}) ---")
    print(f"Initial 8-bit Binary: {bin_str} (Decimal: {int(bin_str, 2)})")
    current = bin_str
    for step in range(1, positions + 1):
        discarded_msb = current[0]
        current = current[1:] + "0"
        print(f"Step {step}: Discard MSB '{discarded_msb}', Append '0' at LSB -> {current} (Decimal: {int(current, 2)})")
    print(f"Final 8-bit Result:   {current} (Decimal: {int(current, 2)})\n")
    return current
def logical_right_shift_8bit(bin_str: str, positions: int = 2):
    print(f"--- Logical Right Shift (>> {positions}) ---")
    print(f"Initial 8-bit Binary: {bin_str} (Decimal: {int(bin_str, 2)})")
    current = bin_str
    for step in range(1, positions + 1):
        discarded_lsb = current[-1]
        current = "0" + current[:-1]
        print(
            f"Step {step}: Discard LSB '{discarded_lsb}', Prepend '0' at MSB -> {current} (Decimal: {int(current, 2)})"
        )

    print(f"Final 8-bit Result:   {current} (Decimal: {int(current, 2)})\n")
    return current
print("================ 8-BIT SHIFT OPERATIONS ================\n")
res_a = logical_left_shift_8bit("10110110", 2)
res_b = logical_right_shift_8bit("11010100", 2)
```

```
iscellaneous programs/2"
================ 8-BIT SHIFT OPERATIONS ========
=======

--- Logical Left Shift (<< 2) ---
Initial 8-bit Binary: 10110110 (Decimal: 182)
Step 1: Discard MSB '1', Append '0' at LSB -> 011
01100 (Decimal: 108)
Step 2: Discard MSB '0', Append '0' at LSB -> 110
11000 (Decimal: 216)
Final 8-bit Result:   11011000 (Decimal: 216)

--- Logical Right Shift (>> 2) ---
Initial 8-bit Binary: 11010100 (Decimal: 212)
Step 1: Discard LSB '0', Prepend '0' at MSB -> 01
101010 (Decimal: 106)
Step 2: Discard LSB '0', Prepend '0' at MSB -> 00
110101 (Decimal: 53)
Final 8-bit Result:   00110101 (Decimal: 53)

(base) vinaya@VINAYAs-MBP miscellaneous programs
% 
```